Inline Arrays in Swift

Introduced in Swift 6.1

BY SHUBAM GUPTA

What is an Inline Array?

Inline Array is a new collection type introduced in **Swift 6.1 (released 2025)**, designed to store a fixed number of elements directly on the **stack** instead of the **heap**.

The syntax looks like this:

```
// Syntax: InlineArray<Count, Element>
var scores: InlineArray<5, Int> = [10, 20, 30, 40, 50]
```

Step-by-Step

1. Understanding Normal Array First

```
Array

var fruits: [String] = ["Apple", "Banana", "Mango"]
fruits.append("Orange")    // ✅ Can grow
fruits.remove(at: 0)       // ✅ Can shrink
```

How it works internally:

Stack —► Reference (pointer)

Heap —► "Apple" "Banana" "Mango" (actual data lives here)

- Lives on the Heap
- Has dynamic size (can grow/shrink)
- Has reference counting overhead (ARC)
- Has performance cost for heap allocation.

2. What Problem Does **Inline Array Solve**?

Before Swift 6.1, if you wanted a **fixed-size**, **stack-allocated** collection you had to use ugly workarounds:

Tuples

```
// Old workaround using Tuples
var scores: (Int, Int, Int, Int, Int) = (10, 20, 30,
40, 50)

// Not iterable, not clean, not readable!
```

3. Inline Array solves this elegantly.

- **No heap**.
- **No pointer**.
- **No ARC**.

Just raw, fast memory.

How it works internally - Stack Only

10 | 20 | 30 | 40 | 50 -- (all data lives right here)

Inline-Array

```swift
// ✅ Clean fixed-size array on the STACK
var scores: InlineArray<5, Int> = [10, 20, 30, 40, 50]

// Accessing elements - same as normal array
print(scores[0])   // 10
print(scores[4])   // 50

// Iterating - works normally
for score in scores {
    print(score)
}
```

Key Differences Table

Feature	Normal Array [T]	InlineArray<N,T>
Memory Location	Heap	Stack
Size	Dynamic (flexible)	Fixed at compile time
ARC Overhead	Yes	No
Performance	Good	Better / Faster
append() / remove()	✅ Supported	❌ Not supported
Introduced In	Since Day 1	Swift 6.1 (2025)
Best For	General use	Performance-critical code

When Should You Use Inline Array?

✅ Use Inline Array when:

- You know the exact count at compile time
- You need maximum performance (game loops, audio buffers, ML)
- Working with low-level system code
- Avoiding heap allocations is critical

Inline-Array

```
// Perfect use cases:
// 1. RGB Color channels (always 3)
var colorChannels: InlineArray<3, Float> = [0.9, 0.4,
0.2]

// 2. 3D coordinates (always x, y, z)
var position: InlineArray<3, Double> = [1.0, 2.5, 3.8]

// 3. Fixed matrix for game/graphics
var matrix: InlineArray<16, Float> = [...]
```

❌ **Stick with Normal Array when:**

- Size changes at runtime
- You need append, remove, insert
- Working with user input or API responses

Limitations to Know

❌ **Stick with Normal Array when:**

- Size changes at runtime
- You need append, remove, insert
- Working with user input or API responses

Inline-Array

```
var scores: InlineArray<5, Int> = [10, 20, 30, 40, 50]

// ❌ These will NOT compile - size is fixed!
scores.append(60)
scores.remove(at: 0)

// ✅ You CAN mutate existing values
scores[2] = 99   // Fine! Just replacing, not resizing
```

Q. Does InlineArray Support Heterogeneous Types?

*InlineArray only accepts **one type** for all elements.*

Q. Why Can't It Hold Mixed Types?

- *It's all about **memory layout** on the Stack.*
- Stack allocation needs exact byte size at compile time.
- Mixed types = unpredictable size = Stack can't handle it.

*This is a great feature for **iOS performance-critical code** - especially things like Metal shaders, CoreML buffers, ARKit transforms, and game engines*

Thank you
By Shubam Gupta